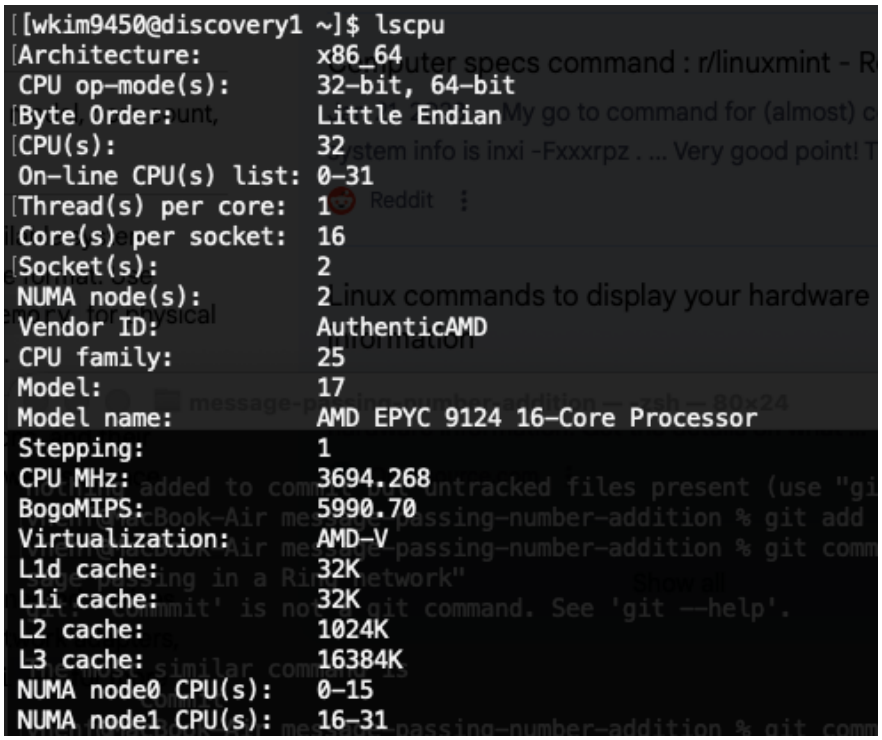
EE-451: Parallel and Distributed Computation

Wooyoung Kim(2717106812) phw4

March 8th 2026

1 Hardware Specification (CARC)



```
[wkim9450@discovery1 ~]$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                32
On-line CPU(s) list:   0-31
Thread(s) per core:    1
Core(s) per socket:    16
Socket(s):             2
NUMA node(s):          2
Vendor ID:             AuthenticAMD
CPU family:            25
Model:                 17
Model name:            AMD EPYC 9124 16-Core Processor
Stepping:              1
CPU MHz:               3694.268
BogoMIPS:              5990.70
Virtualization:        AMD-V
L1d cache:             32K
L1i cache:             32K
L2 cache:              1024K
L3 cache:              16384K
NUMA node0 CPU(s):     0-15
NUMA node1 CPU(s):     16-31
```

Figure 1: Hardware specification for homework

2 Pass Message in a Ring

Implementation

The objective was to implement a message-passing ring using MPI with four distinct processes. Process 0 initializes a message with the value 451 and sends it to Process 1. Each subsequent process receives the message using the blocking `MPI_Recv` function, increments the integer value by 1, prints its current state, and forwards it to the next process in the ring using `MPI_Send`. Finally, Process 3 sends the incremented message back to Process 0, successfully completing the cycle.

Execution & Results

To execute the program on the CARC cluster, a SLURM batch script named `p1_job.sl` was utilized. The script requests 4 tasks and writes the terminal output to a file named `p1_result.out`.

File: `p1_job.sl`

```
#!/bin/bash
#SBATCH --job-name=mpi_ring
#SBATCH --time=01:00:00
#SBATCH --ntasks=4
#SBATCH --output=p1_result.out
make p1
srun ./p1
```

The job was submitted to the cluster queue using the following command:

```
sbatch p1_job.sl
```

The job was executed successfully across compute nodes.

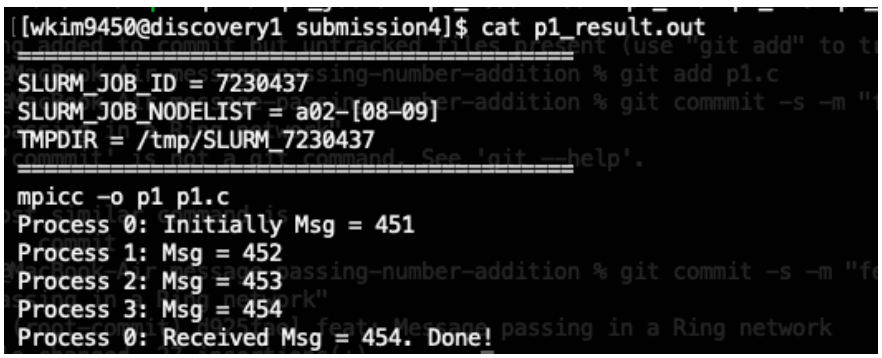


Figure 2: Program Output

3 Add 64 Numbers Using 4 Processes

Implementation

This task involved computing the total sum of 64 integers read from a file (`number.txt`) utilizing four MPI processes. To demonstrate different levels of MPI communication efficiency, the solution was implemented using three distinct approaches:

- **Approach 1:** Each process reads the entire array independently, computes its assigned 16-element partial sum, and manually sends the result to Process 0 using the `MPI_Send` and `MPI_Recv`.
- **Approach 2:** Process 0 reads the array and broadcasts it to all processes using `MPI_Bcast`. After each process computes its partial sum, the `MPI_Reduce` is used with the `MPI_SUM` operator to aggregate the total sum at Process 0.
- **Approach 3:** To optimize memory usage, Process 0 reads the array and distributes exactly 16 elements to each process using `MPI_Scatter`. After the processes compute the sum of their smaller local arrays, Process 0 collects the individual partial sums back into an array using `MPI_Gather` and computes the final sum.

Execution and Results

All three programs were compiled with `mpicc` and executed on the CARC cluster via SLURM batch jobs. As shown in the outputs below, all three communication strategies correctly computed the same total sum of 47126, verifying the accuracy.

Approach 1

File: `p2_1_job.sl`

```
#!/bin/bash
#SBATCH --job-name=p2_1
#SBATCH --time=01:00:00
#SBATCH --ntasks=4
#SBATCH --output=p2_1_result.out
make p2_1
srun ./p2_1
```

The job was submitted to the cluster queue using the following command:

```
sbatch p2_1_job.sl
```

The job was executed successfully across compute nodes.

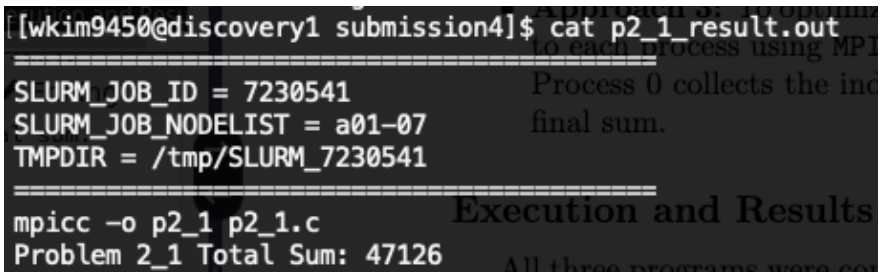


Figure 3: Approach 1 Output

Approach 2

File: `p2_2_job.sl`

```
#!/bin/bash
#SBATCH --job-name=p2_2
#SBATCH --time=01:00:00
#SBATCH --ntasks=4
#SBATCH --output=p2_2_result.out
make p2_2
srun ./p2_2
```

The job was submitted to the cluster queue using the following command:

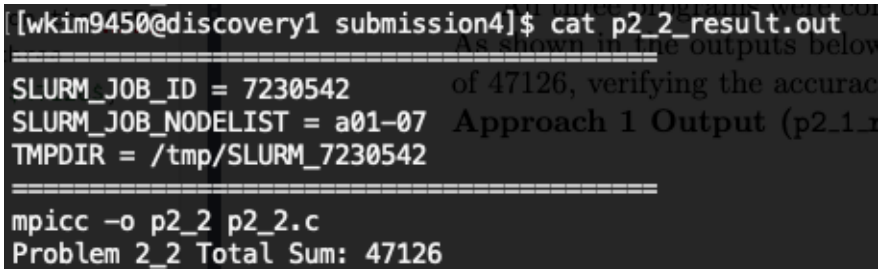
```
sbatch p2_2_job.sl
```

The job was executed successfully across compute nodes.

Approach 3

File: `p2_3_job.sl`

```
#!/bin/bash
#SBATCH --job-name=p2_3
#SBATCH --time=01:00:00
#SBATCH --ntasks=4
#SBATCH --output=p2_3_result.out
make p2_3
srun ./p2_3
```



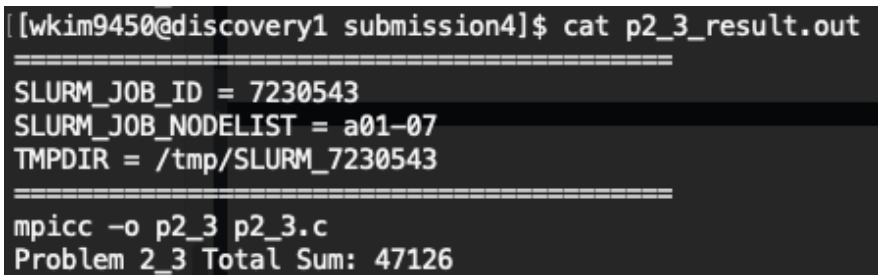
```
[wkim9450@discovery1 submission4]$ cat p2_2_result.out
==========================================
SLURM_JOB_ID = 7230542
SLURM_JOB_NODELIST = a01-07
TMPDIR = /tmp/SLURM_7230542
==========================================
mpicc -o p2_2 p2_2.c
Problem 2_2 Total Sum: 47126
```

Figure 4: Approach 2 Output

The job was submitted to the cluster queue using the following command:

```
sbatch p2_3_job.sl
```

The job was executed successfully across compute nodes.



```
[wkim9450@discovery1 submission4]$ cat p2_3_result.out
==========================================
SLURM_JOB_ID = 7230543
SLURM_JOB_NODELIST = a01-07
TMPDIR = /tmp/SLURM_7230543
==========================================
mpicc -o p2_3 p2_3.c
Problem 2_3 Total Sum: 47126
```

Figure 5: Approach 3 Output